

COMPUTER SCIENCE TRIPOS Part IB – 2022 – Paper 5

4 Concurrent and Distributed Systems (djg11)

Exclusion

- (a) A shared resource is updated by one writer at a time but must support concurrent readers. One or more locks are used to manage exclusion.

- (i) What design choices exist regarding fairness? [2 marks]

Answer: A simple design, where the writer may only enter when there are no readers, may be unfair to writers since readers are always overlapping. An improvement is to stop new readers from entering when a writer is waiting.

- (ii) Compare having two simple locks, respectively used for read and write exclusion, with having a single, specialised locking primitive. What additional state transition(s) could the specialised lock support?

[4 marks]

Answer: Separate mutexes for reading and writing, augmented with a counter for the number of active readers, are the basis of the simple Raynal MRSW implementation, which does not offer fairness. The fair implementation lectured ends up with consecutive invocations of simple primitives which may attract overhead in some systems whereas having the MRSW primitive as a primitive avoids this. Using the specialised MRSW lock, a common useful additional transition is to upgrade from non-exclusive reader state to exclusive writer state.

- (iii) If deadlock is to be avoided using a locking order, is a complete or partial order needed? [2 marks]

Answer: Deadlock is defined to be an instance of a cyclic wait pattern. Cycles can be avoided by placing all locks in a published complete order where no thread waits for a lock on any resource earlier than any lock it is already holding. If nothing about what threads will do is known, a total ordering has to be used. Using a monolithic total order is overly restrictive and prohibits many serializable schedules that may be possible, degrading performance. Since threads do have pre-coded behaviours, many aspects of the total ordering are irrelevant and it effectively behaves as a partial order. Anywhere in the code, if and only if two locks are possibly held by one thread, the order between the locks should be specified, giving us a partial order. *A short answer is fine, but no credit awarded for unjustified answers.*

- (iv) How should the various transitions possible with the specialised lock mentioned above be integrated into a lock ordering policy? [3 marks]

Answer: To advance a lock from read to write state, a thread must touch the lock twice, hence you could suggest it should appear twice in a global ordering. But this leads to cyclic dependencies: when two threads that hold an MRSW lock in read mode want to upgrade it, the system will deadlock. For the third credit point, it can be mentioned that having the separate read and write locks will not have this problem or a solution along the lines of at most one read-with-intention-to-write thread can be allowed. *Further, finer detail is unnecessary, e.g. for any threads A and B, we can only take A's locks before we take B's locks, whether read or write, and we can only promote an A lock to*

an exclusive write lock before we take any lock in B. The demotion operation should be safe because a write lock is strictly stronger than a read lock, so we can leave it out of the ordering policy.

(b) A mutex is acquired at the start of the body of a function.

(i) What can go wrong if the function is recursive? [1 mark]

Answer: In the simple situation of the acquire being unconditional at the entry to the body, the mutex will be held by the parent of a recursive invocation and eternally block the thread and any others that attempt to acquire it.

(ii) To avoid the problem, a friend suggests that threading systems should allow lock acquisitions to proceed if the lock is already held by the same piece of code. Could something like this work and is it worthwhile? [3 marks]

Answer: ‘The same piece of code’ is somewhat vague and would need to be clarified. ‘The same callsite’ would be a poor solution in general since different threads at that site should properly interfere with each other. Ditto the same callpath (list of callsites on stack). Ignoring subsequent lock attempts by the same thread on a given mutex is potentially sensible, but probably a counting system should be invoked so that unlock only happens after the same number of successful acquisitions. Hence the holder identity and a counter need to be added to the mutex structure if not already present. Java’s synchronised classes and the C++ `recursive_mutex` are real-world examples.

Credit can be awarded for arguing that re-entrant locks allow certain APIs to remain the same as before they were made thread-safe. Equally it can be argued that naively allowing mutexes to become re-entrant may defeat the expected atomicity semantics, especially when calling between instances. But the preferred answer is that the additional complexity should perhaps not be added to every mutex and that it is better to modify the control flow or define a wrapper method to acquire a simple mutex at the top of any recursion.

(c) The behaviours provided by a shared, global variable are to be emulated within a message-passing system.

(i) Give pseudocode that provides the emulation. [2 marks]

Answer: There can be a single thread/actor that handles all the read/write requests. This part lays the ground for the next part and is nothing more than setter and getter methods:

```
on receiving (src, dest, "write", v)
    { x := v; send (dest, src, "ack") }
on receiving (src, dest, "read") { send (dest, src, x) }
```

(ii) Discuss whether emulation of locking primitives, such as semaphores, is possible or would be worthwhile? [3 marks]

Answer: Assuming the message-passing system can atomically transfer records of the size that need to be locked, the emulation can be of a global variable of that size

and no further locking may commonly be required. An operation that is equivalent to test-and-set is also easy to implement, should such locking be needed in other situations.

```
on receiving (src, dest, "test-and-set", ov, nv)
{ let p = (x==ov); if (p) x := nv; send (dest, src, p) }
```

Common design patterns, such as claiming the next sequence number from a counter, should ideally not be literally translated to exploit this emulation of global variables and locks. It is much better to properly exploit the message-passing style and implement an agent/actor whose function is to serve fresh identifiers. *Note: this is a question about reliable message passing and consensus algorithms from distributed systems are not needed.*
